第 8 章　排序

8.1 引言

排序（sorting）是我们在计算机数据处理中经常遇到的。特别是，在事务处理中排序占了很大的比重，一般认为其中四分之一以上的时间都用在排序上。排序的功能是将一组杂乱无章的数据元素或记录序列重新排列成一个按关键字有序的序列。

例如，对学生绩点的排名；在任一购物网站上，对信用度、销量等排序；第 7 章中介绍的折半查找能极大提高查找效率，但前提是顺序表按关键字有序。这些都要求对线性表进行排序，因此排序算法是计算机程序设计中的一种重要操作。

8.2 排序的定义与分类

8.2.1 排序的定义

假设有一个含n个记录的序列$\{R_1, R_2, \dots, R_n\}$，其相应的关键字序列为$\{K_1, K_2, \dots, K_n\}$，需确定针对 $1, 2, \cdots, n$的一种排列$p_1, p_2, \dots, p_n$，使其相应的关键字满足非递减（正序）$K_{p_1} \le K_{p_2} \le \cdots \le K_{p_n}$或非递增（逆序）关系，使得记录序列成为一个按关键字有序的序列$\{R_{p_1}, R_{p_2}, \dots, R_{p_n}\}$，这样一种操作称为**排序**。

排序定义中的关键字K既可以是主关键字，也可以是次关键字。若K是主关键字，则任何一个记录的无序序列经排序后得到的结果是唯一的；若K是次关键字，则排序的结果不唯一，因为待排序的记录序列中可能存在两个或两个以上关键字相等的记录。假设$K_i = K_j$，$(1 \le i \le n,\ 1 \le j \le n,\ i \ne j)$，且在排序前的序列中$R_i$在$R_j$之前，(即$i < j$)，若在排序后的序列中$R_i$仍在$R_j$之前，则称所用的排序方法是**稳定的**；反之，若在排序后的序列中R_j在R_i之前，则称所用的排序方法是**不稳定的**。

8.2.2 排序方法分类

根据算法思想，排序一般可分为五类：插入排序、交换排序、选择排序、归并排序和基数排序。根据排序过程的时间复杂度，排序则可分为三类：简单的排序方法，其时间复杂度为$O(n^2)$；改进的排序方法，其时间复杂度为$O(n \log n)$；基数排序，其时间复杂度为$O(d \cdot n)$。

由于排序记录的数据量可能超过内存容量，因此可将排序分为两大类：一类是内部排序，指的是排序记录存放在计算机内存中进行的排序过程；另一类是外部排序，指的是排序记录的数据量大，内存一次不能容纳全部记录，在排序过程中尚需对外存进行访问的排序过程。本章主要讨论内部排序。

内部排序的方法很多，每种方法都有各自的优缺点，因此适合的使用环境（如记录的数据量多少，初始排列状态等）不同。一般评价一个算法的好坏的标准只有两个：时间复杂度和空间复杂度。本章仅就每一类介绍一些典型算法，有兴趣的学生可阅读更多的课外资料，以便于学习和改进算法。

8.2.3 排序的数据类型

常见的排序算法将待排序的一组记录存放在地址连续的一组存储单元中，即采用线性表的顺序存储结构，因此在序列中相邻的两个记录的存储位置也相邻。在这种存储方式中，记录之间的次序关系由其存储位置决定，则实现排序必须借助“比较”和“移动”这两个基本操作。

待排记录的数据类型设为：

```
#define  MAXSIZE  256  // 用作示例的顺序表的最大长度
typedef int  KeyType;      // 示例中的关键字类型为整数类型
typedef  struct
{
    KeyType key;          // 关键字项
    InfoType info;        // 其他数据项
}RecordType;              // 定义记录类型
typedef  struct
{
    RecordType r[MAXSIZE+1]; // 0 号单元闲置
    int length;              // 顺序表长度
}SqList;                     // 用于排序的顺序表类型
```

为了能更好地说明算法思想，在下面介绍的排序算法中，假设记录的关键字均为整数，排序算法默认为实现非递减（正序）排序。如果要实现逆序排序，则在算法中稍作修改即可。

8.3 插入排序

插入排序（insertion sort）的原则是将一个待排序的记录按关键字值的大小插入到前面已经排序好的一组记录的适当位置，直到所有记录完成排序为止。

8.3.1 直接插入排序

直接插入排序（straight insert sort）是一种最简单的排序方法，它的基本操作是将一个记录插入已排好序的有序表，从而得到一个新的、记录数增加 1 的有序表。

例 8.1 直接插入排序示例 设有一个由待排序的一组记录组成的线性表，关键字的初始排列为{ 48，35，66，91，74，18，22，48}。我们可以认为第一个关键字 {48} 组成的线性表已是有序表；将第二个关键字插入前面的有序表，使之有序，得到序列{35，48}；再将第三个关键字插入，使之有序，得到序列{35，48，66}；如此重复的，直到所有关键字都已插入，完成排序（如图 8.1 所示）。

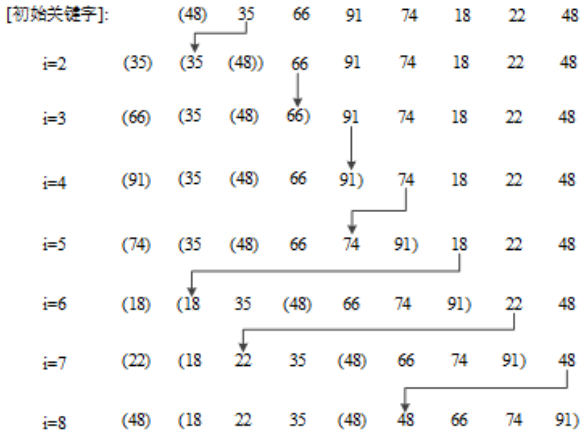


图 8.1 直接插入排序示例

可见，直接插入排序的算法思想如下：

（1）假设由前$i-1$个记录关键字组成的子序列$r[1..i-1]$已为正序，在子序列中寻找合适位置并插入记录关键字$r[i]$，使子序列成为含有i个记录关键字的正序子序列。

（2）对有n个记录关键字的序列，重复上述过程至$i=n$，完成整个序列的直接插入排序。

相应的算法实现如下。

算法 8.1 第 i 个记录的插入

```
void Insert( SqList &L, int i )
{
    // 将记录 r[i]插入到前 i-1 个记录关键字组成的有序子序列，结果保持有序
    temp = L.r[i];                                  // 将第 i 个记录临时保存
    for( j = i; j >1; j-- )
        if( temp.key < L.r[j-1].key ) L.r[j] = L.r[j-1];   // 比较关键字，若待排关键字小，则后移记录
        else break;                                  // 已确定插入位置
    L.r[j] = temp;                                   // 记录 r[i]插入到正确位置
}// Insert
```

算法 8.2 直接插入算法

```
void InsertSort( SqList &L)
{
    //对顺序表 L 作直接插人排序
    for( i=2; i<=L.length; i++ )                    //从第 2 个记录开始，完成插入排序
        Insert(L,i);
} //InsertSort
```

接下来对直接插入排序算法进行分析。

（1）时间复杂度分析：对于算法 8.1，前$i-1$记录关键字已为正序，如果$r[i] \geq r[i-1]$，则是最好的情况，无需移动记录；如果$r[i] < r[1]$，则是最坏的情况，要移动$i-1$个记录。在大多数情况下，排序记录是随机的，可能出现的各种排列的概率相同，因此该算法需要平均移动一半的记录，即$(i-1)/2$。算法 8.2 从第二个记录开始实现插入，直到最后一个记录，平均移动次数为$\sum_{i=2}^{n}\frac{i-1}{2}=n(n-1)/4$，因此$T(n)=O(n^2)$。

（2）空间复杂度分析：在数据交换过程中，该算法只用到一个临时存储空间 temp，因此$S(n)=O(1)$。

（3）稳定性分析：在插入过程，该算法只对连续位置的记录做比较交换，关键字相同记录不交换，因此该算法是稳定的。

（4）改进策略：在$r[i]$确定插入位置时，可以对前面$i-1$个有序子序列进行折半查找，确定位置后再进行向后移位，这样可以减少“比较”的次数，但“移动”的次数不变。由于排序算法的主要工作量在移位操作上，所以算法的时间复杂度不变$T(n)=O(n^2)$ 。

8.3.2 希尔排序

希尔排序（Shell' s sort）又称“缩小增量排序”（ diminishing increment sort），是一种改进的插人排序算法，在时间效率上有较大的提高。

通过对直接插入排序的分析可知，其算法时间复杂度为$O(n^2)$，当待排记录序列为有序时，其时间复杂度可提高至$O(n)$。当待排记录序列按关键字“基本有序”时，插入排序的效率就可大幅提高。从另一方面来看，由于直接插入排序法很简单，所以它在n值很小时的效率也比较高。希尔排序正是从这两点分析出发，对直接插入排序进行改进。

希尔排序的基本操作过程是：先将整个待排记录序列分成为若干个子序列，分别进行直接插入排序，待整个序列中的记录“基本有序”时，再对全体记录进行一次直接插入排序。设d为子序列的增量，即在记录序列中，每间隔d个记录取一个记录，组成记录子序列。例如进行 3 趟排序过程：第 1 趟，$d=5$，分别以第 1、2、3、4、5 个记录为起点，每间隔 5 个记录取一个记录，组成 5 个记录子序列，然后在原来记录的存储位置上分别对这 5 个记录子序列进行直接插入排序；在上述的基础上，第 2 趟令$d=3$，进行 3 个子序列的直接插入排序； 最后一趟令 $d=1$，即对所有记录进行直接插入排序。d值即为子序列记录间隔的增量，每次排序过程的增量逐渐缩小，直至最后一次$d=1$，完成整个序列的排序。

例 8.2 希尔排序示例 以关键字序列{ 48，35，66，91，74，18，22，48，57，3} 为例，d 分别取 5、3、1，希尔排序的过程如图 8.2 所示。

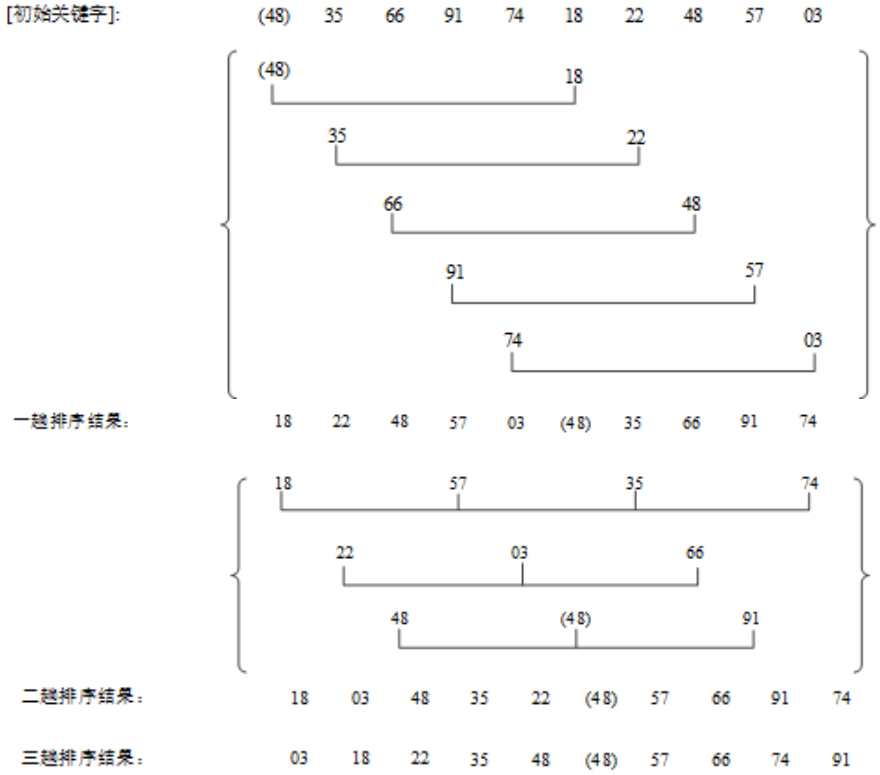


图 8.2 希尔排序示例

从上述排序过程可见，希尔排序的特点是子序列的构成不是简单地将序列“逐段”分割，而是将相隔某个“增量”的记录组成一个子序列。如上例中，第 1 趟排序时的增量为 5，第 2 趟排序时的增量为 3。由于在原来的记录存储位置上，在插入排序中记录的关键字是和同一子序列中的前一个记录的关键字进行比较，因此关键字较小的记录就不是一步一步地往前挪动，而是跳跃式地往前移，从而使得在进行最后一趟增量为 1 的插入排序时，序列已基本有序，只要进行记录的少量比较和移动即可完成排序。因此希尔排序的时间复杂度比直接插入排序低。

综上，希尔排序算法思想如下：

（1）对间隔为d的子序列进行直接插入排序；

（2）逐步缩小增量的取值d，重复上述过程，直到d为 1，完成整个序列的希尔排序。

相应的算法实现如下。

算法 8. 3 增量为 d 时的直接插入排序

```
void ShellInsert( SqList &L, int d )
{    // 对间隔 d 的子序列进行直接插入排序
    for( i=d+1; i<=L.length; i++ )
```

```
    {
        if( L.r[i].key <  L.r[i-d].key )
        {    // 需将 L.r[i]插入有序增量子表
             temp =  L.r[i];         // 将第 i 个记录临时保存
             for( j=i; j>d; j=j-d )
             {
                 if( temp.key < L.r[j-d].key ) L.r[j]= L.r[j-d];    // 比较关键字并后移记录
                   else break;
             }  // end for j
             L.r[j] = temp;        // 记录 r[i]插入到正确位置.8
        } // end if
   } // end for i
}// ShellInsert
```

算法 8.4 希尔排序算法

```
void ShellSort ( SqList &L, int d[], int t )
{     // 按增量序列 d[0..t-1]，对顺序表 L 进行希尔排序
      for ( k=0; k<t; k++ )
           ShellInsert(L, d[k] )；  // 一趟增量为 d[k]的插入排序
}// ShellSort
```

接下来对希尔排序算法进行分析。

（1）时间复杂度分析：希尔排序的耗时分析是一个复杂的问题，因为它消耗的时间与所取“增量”有关，而目前为止尚未求得一种最好的增量序列，但大量的研究已得出一些局部的结论。当d为某个特定取值函数时，希尔排序的时间复杂度为$T(n) = O(n^{1.5})$。但需注意的是，应使增量序列中的值没有除 1 之外的公因子，并且最后一个增量值必须等于 1。

（2）空间复杂度分析：在数据交换过程中，该算法始终只用到一个临时变量，因此 $S(n) = O(1)$。

（3）稳定性分析：在子序列插入过程，由于发生了两个相隔增量位置的记录交换，相同关键字记录的次序可能也被改变，因此算法是不稳定的。

8.4 交换排序

交换排序（exchange sort）是一种常用的排序思想，它通过比较两个记录关键字并交换位置得到最终的排序结果。

8.4.1 简单交换排序

简单交换排序也称为**冒泡排序**（bubble sort）。它的基本操作过程很简单。首先将第 1 个记录的关键字和第 2 个记录的关键字进行比较，若为逆序，则将两个记录交换，然后比较第 2 个记录和第 3 个记录的关键字，以此类推，直至第$n-1$个记录和第n个记录的关键字进行过比较为止。这个过程称作第 1 趟冒泡，其结果使得关键字最大的记录被安置到最后一个记录的位置上。然后进行第 2 趟冒泡，对前$n-1$个记录进行同样的操作，其结果是关键字次大的记录被安置到第$n-1$个记录的位置上。以此类推，直至完成第$n-1$趟冒泡，即完成排序。

例 8.3 冒泡排序示例 以关键字序列{ 48，35，66，91，74，18，22，48，57，03 }为例，冒泡排序的过程如图 8.3 所示。

[初始关键字]:	(48)	35	66	91	74	18	22	48	57	03
第一趟排序后：	35	(48)	66	74	18	22	48	57	03	**91**
第二趟排序后：	35	(48)	66	18	22	48	57	03	**74**	**91**
第三趟排序后：	35	(48)	18	22	48	57	03	**66**	**74**	**91**
第四趟排序后：	35	18	22	(48)	48	03	**57**	**66**	**74**	**91**
第五趟排序后：	18	22	35	(48)	03	**48**	**57**	**66**	**74**	**91**
第六趟排序后：	18	22	35	03	**(48)**	**48**	**57**	**66**	**74**	**91**
第七趟排序后：	18	22	03	**35**	**(48)**	**48**	**57**	**66**	**74**	**91**
第八趟排序后：	18	03	**22**	**35**	**(48)**	**48**	**57**	**66**	**74**	**91**
第九趟排序后：	**03**	**18**	**22**	**35**	**(48)**	**48**	**57**	**66**	**74**	**91**

图 8.3　冒泡排序示例

冒泡排序的算法思想总结如下：

（1）对相邻记录关键字进行两两比较，通过交换将较大者向后移动，每一趟排序将最大者移到最后。

（2）第 1 趟，n个记录参加比较，得到最大值；第 2 趟，前$n-1$个记录参加比较，得到最大值；以此重复，直到仅剩一个记录，完成排序。

冒泡排序的算法实现如下。

算法 8.5 冒泡排序算法

```
void BubbleSort( SqList &L )
{   // 对顺序表 L 进行冒泡排序
    for( i=1; i<=L.length-1; i++ )                          // 进行 n-1 次冒泡过程
    {
        for( j=1; j<=L.length-i; j++ )                      // 前 n-i 个关键字两两比较
        {
            if( L.r[j].key > L.r[j+1].key )                 // 比较关键字
            {
                temp=L.r[j]; L.r[j]= L.r[j+1]; L.r[j+1]=temp;   // 逆序则交换，使大者后移
             } // end if
        } // end for j
    } // end for i
}// BubbleSort
```

接下来对冒泡排序算法进行分析。

（1）时间复杂度分析：算法的第 1 趟进行了$n-1$次记录关键字的比较，平均移动一半的记录，即$(n-1)/2$；第 2 趟比较了$n-2$个记录……直到最后一个记录的平均移动次数为$\sum_{i=1}^{n-1}\frac{n-i}{2}=n(n-1)/4$。因此无论记录关键字是有序还是无序，时间复杂度不变$T(n)=O(n^2)$。

（2）空间复杂度分析：在数据交换过程中，该算法只用到一个临时变量，因此$S(n)=O(1)$。

（3）稳定性分析：在交换过程中，该算法只对连续位置的记录做比较交换，关键字相同记录不交换，因此该算法是稳定的。

（4）改进策略：在冒泡排序的每趟排序中，为了得到当前的最大数，进行了大量的与之前重复的比较交换，如果减少重复的比较交换次数，算法的效率将得到提高。因此，对应的改进策略可为，在一趟冒泡过程中，如果没有出现记录交换，则认为已完成排序，可以结束排序。

8.4.2 快速排序

快速排序（ quick sort）是对冒泡排序的一种改进，属于交换排序。它的基本思想是，通过一趟排序过程，将待排记录分割成独立的两部分，使其中一部分记录的关键字均比另一部分记录的关键字小，再分别对这两部分记录继续递归地进行排序，最终达到整个序列有序。

假设有一个由n个记录组成的待排序的序列。首先任意选取一个记录（可选第一个记录L.r[1]）作为枢轴（ pivot），按下述原则重新排列其余记录：将所有关键字比它小的记录都移到它的位置之前，将所有关键字比它大的记录都移到它的位置之后。此时，该“枢轴”记录已位于排序后的位置。最后将该“枢轴”记录的位置i作分界线，将序列分割成两个子序列。这个过程称为一趟快速排序。接下来对所分割的子序列递归地进行快速排序，直到子序列只有一个记录为止。

例 8.4 一趟快速排序示例 以关键字序列{ 48，35，66，91，74，18，22，48，57，03 }为例，一趟快速排序的过程如图 8.4 所示。

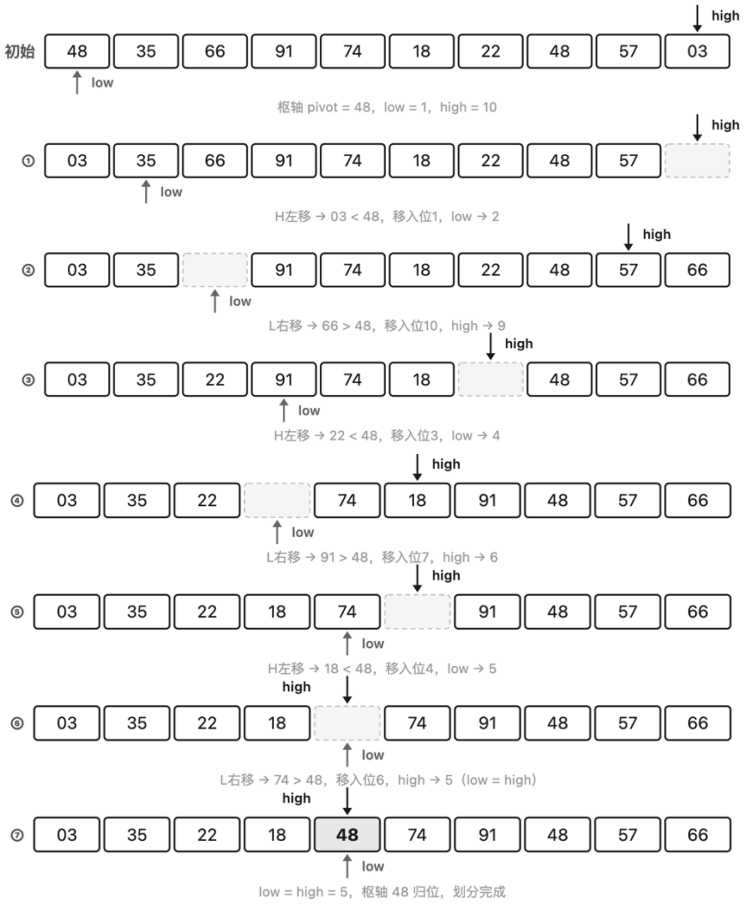


图 8.4　一趟快速排序示例

快速排序的算法思想如下：

（1）一趟快速排序的实现过程：设两个指针 low 和 high，它们的初值分别为子序列的起点和终点。设枢轴记录为第一个记录，并存储在临时变量中，对应关键字称为 pivotkey。首先从 high 所指位置起向前搜索，找到第一个关键字小于 pivotkey 的记录，将其移送到 low 的位置。然后从

low 所指位置起向后搜素，到第一个关键字大于 pivotkey 的记录，将其移送到 high 的位置。重复这两步直至 low=high 为止，将枢轴记录存入交界位置 low，使枢轴记录到位。

（2）对枢轴记录前后两子序列递归地进行快速排序，直到子序列只有一个记录为止。

快速排序的算法实现如下。

算法 8.6 一趟快速排序算法

```
int Partition ( SqList &L, int low, int high )
{   // 一趟快速排序：使枢轴记录到位，并返回其所在位置，此时在它之前(后)的记录均不大(小)于它
    temp = L.r[low];                    // 用子表的第一个记录作枢轴,并存储在临时变量中
    pivotkey= L.r[low].key;             // 枢轴记录关键字
    while( low<high )
    {    // 从表的两端交替向中间扫描
        while( low<high && L.r[high].key>=pivotkey )  high--;
        L.r[low] = L.r[high];           // 将比枢轴记录小的记录移到前端
        while( low<high && L.r[low].key<=pivotkey )   low++;
        L.r[high] = L.r[low];           // 将比枢轴记录大的记录移到后端
    } // end while ( low<high )
    L.r[low] = temp;                    // 枢轴记录到位
    return  low;                        // 返回枢轴位置
} // Partition
```

算法 8.7 low 和 high 区间的快速排序算法

```
void QSort( SqList &L,  int low,  int high )
{   // 在对记录序列进行一趟快速排序后，分别对两个子序列进行递归，完成快速排序
    if ( low<high )
    {
        pivotloc = Partition(L, low, high);   // 一趟快速排序
        QSort(L, low, pivotloc-1);            // 对前部子序列进行递归快速排序
        QSort(L, pivotloc+1,high);            // 对后部子序列进行递归快速排序
    }
}// QSort
```

算法 8.8 快速排序算法

```
void QuickSort( SqList &L )
{   // 对顺序表 L 作快速排序
    QSort(L, 1, L.length);
} // QuickSort
```

接下来对快速排序算法进行分析。

（1）时间复杂度分析：假设对n个记录进行快速排序，k为枢轴记录的位置，则$T(n) = T_{partition}(n) + T(k-1) + T(n-k)$， 其中$T_{partition}(n) = O(n)$，为对$n$个记录进行一趟快速排序的时间复杂度，$T(k-1)$和$T(n-k)$分别为对前后两个子序列进行快速排序 QSort(L,1，k-1)和 QSort(1，k+1，n)的所需时间。假设待排序列中的记录是随机排列的，可以得到$T(n) = O(n\log n)$。

（2）空间复杂度分析：该快速排序算法采用递归实现，在一趟快速排序中，记录交换需要一个临时变量，子序列分别递归，直到递归结束才释放临时变量。因此参照时间复杂度的分析方法，可以得到$S(n) = O(\log n)$。

（3）稳定性分析：在一趟快速排序过程中，前后端的记录与枢轴记录比较产生了前后大距离移动，相同关键字记录的次序可能被改变，因此该算法是不稳定的。

（4）改进策略：快速排序被认为是在同时间复杂度的排序方法中平均性能最好的方法。但是，在初始记录序列按关键字有序或基本有序时，快速排序将退化为冒泡排序，其时间复杂度为

$O(n^2)$。为了对此进行改进，通常以“三者取中”的法则来选取枢轴记录，即在 low、(low+high)/2、high 这三者中，取其关键字中值的记录为枢轴。采用此规则可大幅改善快速排序在最坏情况下的性能。

8.5 选择排序

选择排序（selection sort）是指每一趟排序在剩余的未排序记录中选取关键字最小的记录，将其放置到前一趟有序序列后，有序序列长度增加 1，直到整个序列成为有序序列为止。

8.5.1 简单选择排序

简单选择排序（simple selection sort）的每一趟操作都对剩余的记录做一一做比较，找出最小关键字，并放置到相应的有序位置上。

例 8.5 简单选择排序示例 以关键字序列{ 48，35，66，91，74，18，22，48 }为例，简单选择排序的过程如图 8.5 所示。

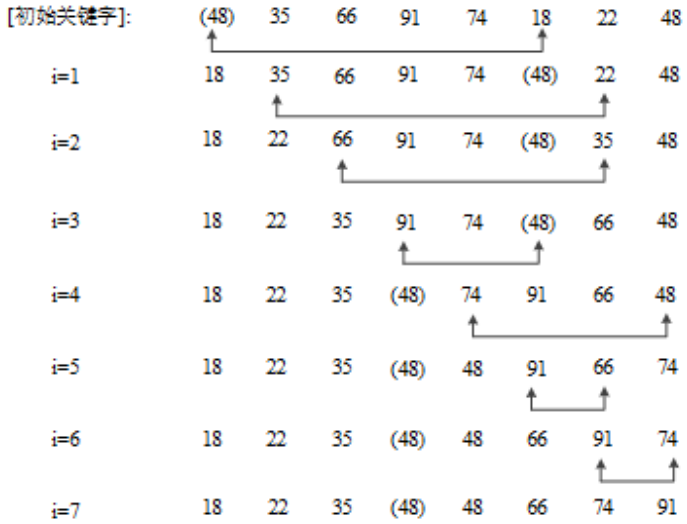


图 8.5　简单选择排序示例

简单选择排序的算法思想如下：

（1）在第i趟排序过程中，通过$n-i$次关键字间的比较，从剩余的$n-i+1$个记录中选出关键字最小的记录，并和第i个记录交换。

（2）对于$i=1,\ldots,n-1$，执行上述过程，直至整个序列有序。

相应的算法实现如下。

算法 8.9 简单选择排序算法

```
void SelectSort (SqList &L)
{   // 对顺序表 L 进行简单选择排序
    for ( i=l; i<L.length;  i++ )
    {  // 共 n-1 趟，选择最小的记录，并交换到位
       min = L.r[i].key;
       k = i;                              // 假设起始记录关键字为最小者
       for ( j=i+1;  j<=L.length;  j++ ) // 与后面记录比较，选择最小的记录
       {
            if ( L.r[j].key<min )          // 发现更小者并存储
            {
                min=L.r[j].key;  k=j;
             } // end if
```

```
        } // end for j
        temp= L.r[i];  L.r[i]= L.r[k]; L.r[k]=temp;   // 找到最小者并交换
    } // end for i
} //SelectSort
```

接下来对简单选择排序算法进行分析。

（1）时间复杂度分析：容易看出，简单选择排序过程中，所需进行的记录移动的操作次数较少。然而，无论记录的初始排列如何，所需进行的关键字间的比较次数都相同，均为 n(n-1)/2。因此$T(n) = O(n^2)$。

（2）空间复杂度分析：在数据交换过程中，该算法只用到一个临时变量，因此$S(n) = O(1)$。

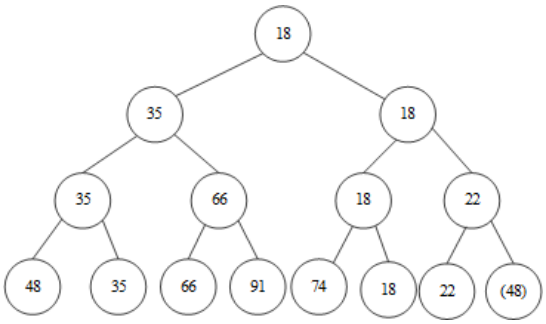
（3）稳定性分析：在每一趟排序过程中，选择最小者并与起始记录交换会产生前后大距离记录交换，相同关键字记录的次序可能被改变，因此该算法是不稳定的。

（4）改进策略：从上述描述可见，选择排序的主要操作是进行关键字间的比较，因此改进简单选择排序应考虑如何减少“比较”。在 n 个关键字中选出最小值至少需要进行 n-1 次比较，但继续在剩余的 n-1 个关键字中选择次小值并非一定要进行 n-2 次比较。因此若能利用前 n-1 次比较所得信息，则可减少以后各趟选择排序中所用的比较次数。

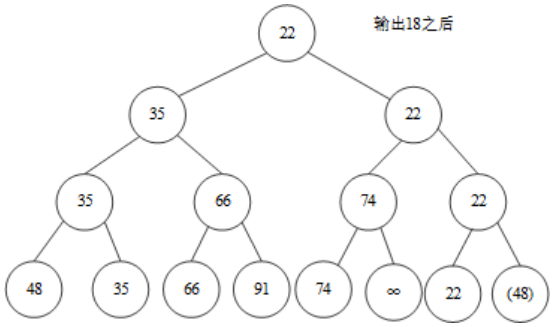
8.5.2 树形选择排序

实际上，体育比赛中的锦标赛便是一种选择排序，称为树形选择排序（tree selection sort），又称锦标赛排序(tournament sort)，是按照锦标赛的思想进行选择排序的方法。首先对 n 个记录的关键字进行两两比较，然后在其中的较小者之间进行两两比较，如此重复，直至选出最小关键字的记录。

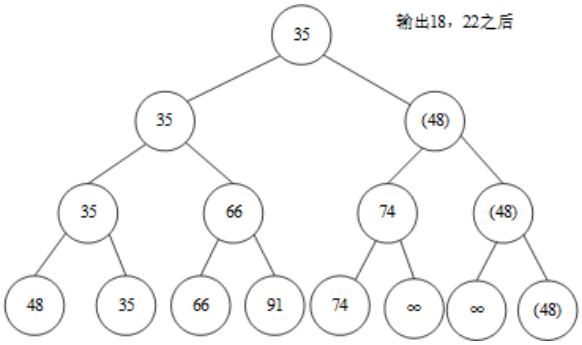
例 8.6 树形选择排序示例 以关键字序列{ 48，35，66，91，74，18，22，48}为例，这个过程可用一棵有 n 个叶子结点的完全二叉树表示，如图 8.6a。这棵二叉树表示了从 8 个关键字中选出最小关键字的过程。8 个叶子结点中依次存放排序之前的 8 个关键字，每个非终端结点中的关键字均等于其左、右孩子结点中较小的关键字，则根结点中的关键字即为所有关键字中的最小者。在输出最小关键字之后，若欲选出次小关键字，则仅需将叶子结点中的最小关键字（18）改为“最大值”，然后从该叶子结点开始，和其左(或右)兄弟的关键字进行比较，修改从叶子结点到根的路径上各结点的关键字，则根结点的关键字即为次小关键字。同理，可依次选出从小到大的所有关键字，参见图 8.6b 和图 8.6c。由于含有 n 个叶子结点的完全二叉树的深度为$\log_2 n + 1$，则在树形选择排序中，除了最小关键字之外，每选择一个次小关键字仅需进行$\log_2 n$次比较，因此它的时间复杂度为$O(n\log_2 n)$。但这种排序方法辅助存储空间较多。



a) 选出最小关键字为 18



b) 选出次小关键字为 22



c) 选出居第三的关健字为 35

图 8.6　树形选择排序示例

8.5.3 堆排序

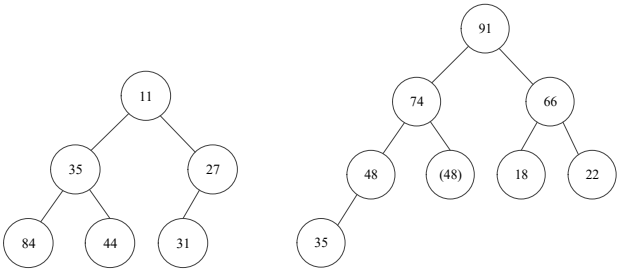
1964 年，威廉姆斯（J. Williams）和弗洛伊德（Floyd）在树形选择排序基础上提出了另外一种选择排序，称为堆排序。**堆排序**（ heap sort）只需要一个记录大小的辅助空间，其比较的中间结果不占用空间，且易于实现。

堆的定义：由n个元素组成的顺序序列$\{k_1, k_2, \ldots, k_n\}$，当且仅当满足以下关系时，称之为堆。

若$k_i \geq k_{2i}$且$k_i \geq k_{2i+1}$，则称为大顶堆；若$k_i \leq k_{2i}$且$k_i \leq k_{2i+1}$，则称为小顶堆。其中$i = 1, 2, \ldots, n/2$。

将关键字序列对应的顺序存储结构（可理解为一维数组）看成是一个完全二叉树，则堆的定义表明，完全二叉树中所有非终端结点的值均不大于（或不小于）其左、右孩子结点的值。由此，若序列$\{k_1, k_2, \ldots, k_n\}$是堆，则堆顶元素（完全二叉树的根）必为序列中$n$个元素的最小值（或最大值）。若堆顶元素为最小者，则该堆称为小顶堆；若堆顶元素为最大者，则该堆称为大顶堆。

例 8.7 堆的示例 例如，下列两个序列（11，35，27，84，44，31）和（91，74，66，48，48，18，22，35）为堆，对应的完全二叉树如图 8.7 所示。



a) 小顶堆　　b) 大顶堆

图 8.7　堆的示例

那么，如何进行堆排序？

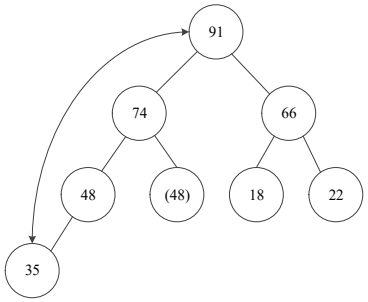
首先将堆顶的最大值与当前最后一个元素交换，这样最大者正好存储在正序序列的最终位置上。再将剩余$n-1$个元素的序列重新调整成一个堆，则得到$n-1$个元素中的最大值。如此反复执行，便能得到一个正序序列。这个过程称为堆排序。

因此，实现堆排序需要解决两个问题：（1）如何由一个无序序列建成一个堆？（2）如何在输出堆顶元素之后，调整剩余元素成为一个新的堆？

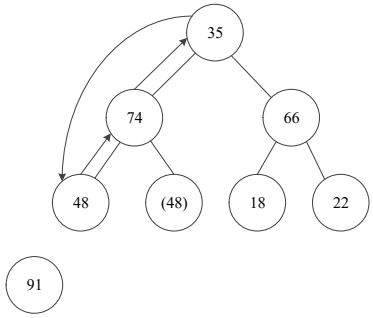
下面首先讨论第二个问题：如何在输出堆顶元素之后，调整剩余元素成为一个新的堆？

例 8.8 堆调整示例 图 8.8a 是一个大顶堆。假设输出堆顶元素之后，以堆中最后一个元素替代之，如图 8.8b 所示。此时根结点的左、右子树均为大顶堆，仅需对根结点关键字自上至下进行调整。首先以堆顶元素和其左、右子树根结点的值大者比较，由于左子树根结点的值大于右子树根

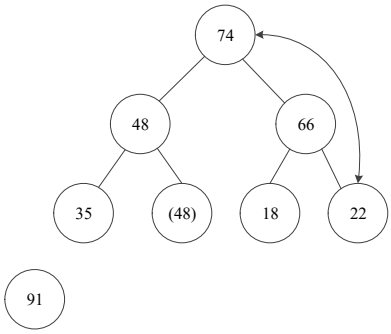
结点的值且大于根结点的值，则将 35 和 74 交换；由于 35 替代 74 之后破坏了左子树的“堆”，所以需进行和上述相同的调整，直至到达叶子结点。调整后的状态如图 8.8c 所示，此时堆顶为 n-1 个元素中的最大值。重复上述过程，将堆顶元素 74 和堆中最后一个元素 22 交换且调整，得到如图 8.8d 所示新的大顶堆。我们称这个自堆顶至叶子的调整过程为“**筛选**”。



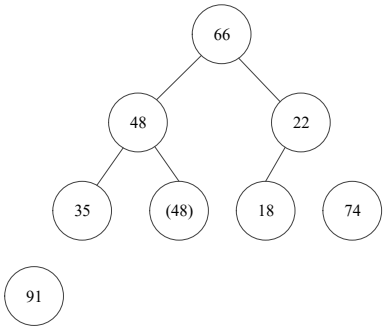
a）大顶堆



b）91 和 35 交换之后的情形



c）建成的新堆



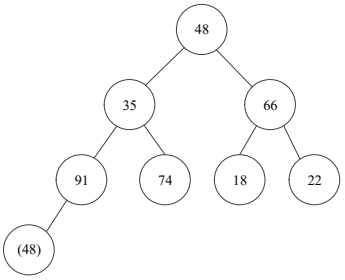
d）22 和 74 交换之后的建成的新堆

图 8.8 堆调整示例

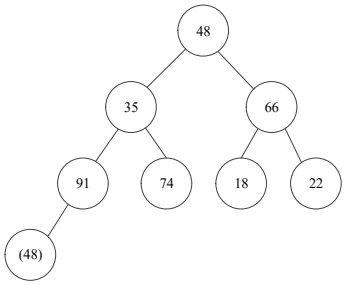
接下来讨论第一个问题：如何由一个无序序列建成一个堆?

实际上，从一个无序序列建堆的过程也是一个反复“筛选”的过程。若将此序列看成是一个完全二叉树，则每个叶子结点为根的子树可以认为是堆。最后一个非终端结点是第$n/2$个元素，因此只需从第$n/2$个元素开始至根结点进行“筛选”。

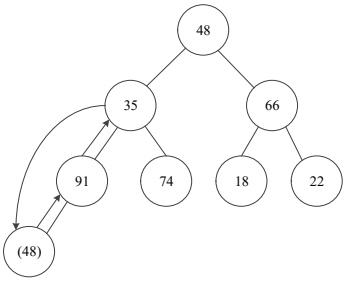
例 8.9 建堆示例 图 8.9a 中的二叉树表示一个有 8 个元素的无序序列{48，35，66，91，74，18，22，48}。则从第 4 个元素开始筛选，由于 91 无右子树并且大于左子树的值，所以筛选后的序列不变，如图 8.9b 所示。同理，在第 3 个元素 66 被筛选之后，序列也不发生改变，此时的状态如图 8.9b 所示。由于第 2 个元素 35 小于 74 和 91，所以要将 35 的位置和 91 交换，又由于交换后的 35 小于 48，所以继续将 35 的位置与 48 交换，筛选过程如图 8.9c 所示。图 8.9e 所示为筛选根元素 48 之后建成的堆。



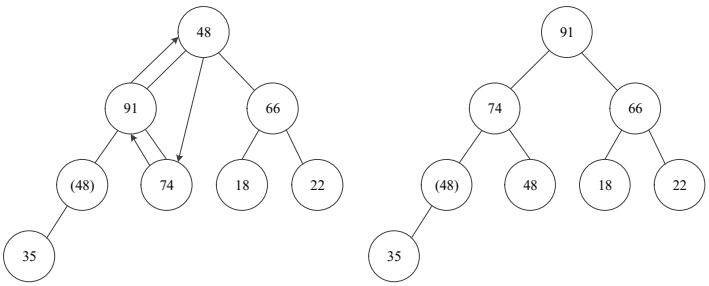
a）初始无序状态



b）91、66 被筛选



c）35 被筛选



d）48 被筛选　　e）建成大顶堆

图 8.9　建堆示例

综上，堆排序的算法思想如下：

（1）在顺序结构的[low, high]范围内，实现“筛选”算法。设 low 为根结点，其左、右子树均已是大顶堆，将根结点值与左、右子树根结点大者进行比较：若较之更大或相等，则已是大顶堆；否则，将根结点值与大者交换，并继续向下比较，直到已是大顶堆为止。

（2）从第$n/2$个元素开始至根结点进行“筛选”，建大顶堆。

（3）将堆顶与当前最后一个记录交换，去除当前最后一个记录，从堆顶进行“筛选”，建立新大顶堆，并重复此过程，直到完成排序为止。

堆排序的算法实现如下。

算法 8.10　堆调整算法

```
void HeapAdjust( SqList &L, int low, int high )     // 堆采用顺序表存储表示
{    // 假设 L 中记录的关键字除 L.r[low].Key 之外，左、右子树均满足大顶堆的定义，进行堆的“筛选”
     temp = L.r[low];
     i = low;  j = 2*i;
     while( j<=high )
     {    // 向下“筛选”
          if (  j+1<=high && L.r[j].key< L.r[j+1].key)  j++;   // 在左、右子树选大者
          if ( temp.key >= L.r[j].key )  break;                // 已是大顶堆
          L.r[i] = L.r[j];  i=j;  j=2*i;                       // 大者上移，并继续向下“筛选”
     }
     L.r[i] = temp;                                            // 插入最终位置
} // HeapAdjust
```

算法 8.11　堆排序算法

```
void HeapSort(SqList &L)
{    // 对顺序表 H 进行堆排序
     for ( i=L.length/2; i>0; i-- )                            // 把 H.r[1..L.1ength]建成大顶堆
          HeapAdjust( L, i, L.length );
     for( i=L.length; i>l; i--)                                // 逐个输出，并调整堆
     {
          temp=L.r[1]; L.r[1]= L.r[i]; L.r[i]=temp;            // 将堆顶记录和当前最后一个记录交换
          HeapAdjust(L, 1,i-1);                                // 将[1..i-1]重新调整为大顶堆
     }
} // HeapSort
```

继续对堆排序算法进行分析。

（1）时间复杂度分析：堆排序方法对记录数较多的情况很有效的。因为其运行时间主要耗费在建初始堆和在调整建新堆时进行的反复“筛选”上，“筛选”是按照树的深度向下进行比较的，而包含n个结点的完全二叉树的深度为$\log_2 n + 1$，所以整个排序过程（包括建堆和调整建新堆）的时间复杂度$T(n) = O(n \log_2 n)$。在最坏的情况下，堆排序的时间复杂度也为$O(n \log_2 n)$，相对于快速排序，这是堆排序的最大优点。

（2）空间复杂度分析：堆排序仅需一个辅助存储空间供交换使用，因此 $S(n) = O(1)$。

（3）稳定性分析：在堆排序过程（包括建堆和调整建新堆）中，会产生前后大距离记录交换，相同关键字记录的次序可能被改变，因此算法是不稳定的。

8.6 归并排序

归并排序（merging sort）是将两个或两个以上的有序序列组合成一个新的有序序列。它的实现方法简单，无论是采用顺序存储结构还是链表存储结构，都易于实现。

设初始序列含有n个记录，则可看成是n个有序的子序列，每个子序列的长度为 1。然后两两归并，得到$n/2$个长度为 2（剩余的不变）的有序子序列；再两两归并。如此重复，直至得到一个长度为 n 的有序序列为止，这种排序方法称为 **2 路归并排序**。

例 8.10 2 路归并排序示例 以关键字序列{48，35，66，91，74，18，22，48，57，03}为例，2 路归并排序的过程如图 8.10 所示。

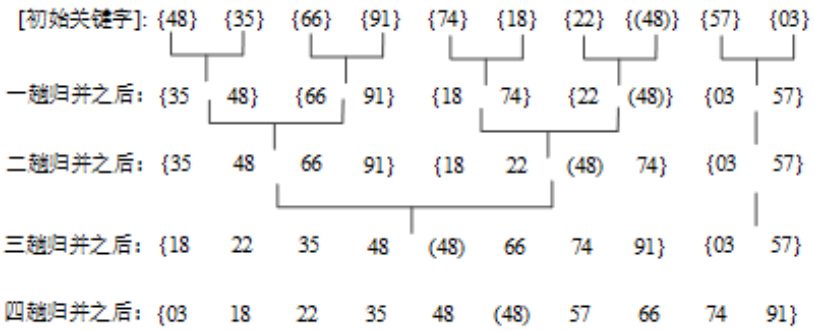


图 8.10 2 路归并排序示例

可见，2 路归并排序算法思想如下：

（1）将存储记录的一维数组S中前后相邻的两个有序序列，归并为一个有序序列T。

（2）将原记录序列的每个记录看成长度为 1 的有序子序列，两两归并。如此重复，直到归并完成为止。

相应的 2 路归并排序的递归代码如下。

算法 8.12 2 路归并过程

```
void Merge( RecordType S[], RecordType &T[], int low, int mid, int high )
{   // 将有序的 S[low..mid]和 S[mid+1..high]归并为有序的 T[low..high]
    i=low; j=mid+1;  k=low;
    while ( i<=mid && j<=high )
    {   // 将 S 中记录由小到大并入 T
        if ( S[i].key<=S[j].key ) { T[k]=S[i]; i++; k++; }
        else { T[k]=S[j]; j++; k++; }
    }
    while( i<=mid ) { T[k]=S[i]; i++; k++; }   // 将剩余的 S[i..mid]复制到 T
    while( j<=high) { T[k]=S[j]; j++; k++; }   // 将剩余的 S[j..high]复制到 T
} // Merge
```

算法 8.13 2 路归并的递归调用算法

```
void MSort( RecordType &S[],RecordType &T[], int low, int high )
{   //将 S[low..high]归并排序成为有序的
    if( low==high )  T[low]=S[low];
    else
    {
        mid = (low+high)/2;          // 将 S[low..high]平分为 S[low..mid]和 S[mid+1..high]
        MSort( S,T,low,mid );        // 递归将 S[low..mid]归并为有序的 T[low..mid]
```

```
            MSort( S,T,mid+1,high );      // 递归将 S[mid+1..high]归并为有序的 T[mid+1..high]
            Merge( T, S, low, mid, high ); // 将 T[low..mid]和 T[mid+1..high]归并到 S[low..high]
        } // end else
    } // MSort
```

算法 8.14 2 路归并算法

```
void MergeSort( SqList &L )
{    // 对顺序表 L 作归并排序
     SqList T;
     MSort(L.r, T.r, 1, L.length);
} // MergeSort
```

2 路归并排序的递归算法在形式上较为简洁，但递归过程占用大量的辅助空间，实用性差，通常采用非递归算法。

2 路归并排序的非递归代码如下。

算法 8.15 非递归 2 路归并过程

```
void Merge( SqList &L, int low, int mid, int high )
{    // 将两个有序表 L[low..mid]和 L[mid+1....high]归并为一个有序表 L[low...high]

        SqList T;
        i=low; j=mid+1;  k=0;
        while( i<=mid && j<=high)   // L 中记录由小到大地并入 T
        {
           if( L.r[i].key<=L.r[j].key ) { T.r[k]=L.r[i]; i++; k++;  }
           else { T.r[k]=L.r[j]; j++; k++;  }
        }
        while( i<=mid ) { T.r[k]=L.r[i]; i++; k++; }     // 将剩余的 L[i..mid]复制到 T
        while( j<=high ) { T.r[k]=L.r[j]; j++; k++; }    // 将剩余的 L[j..high]复制到 T
        for( k=0,i=low; i<=high; k++,i++)                // 合并结果送回原空间
            L.r[i]=T.r[k];
}// Merge
```

算法 8.16 2 路归并的非递归调用算法

```
void MSort( SqList &L, int len )
{    // 实现一趟归并，将两两长度为 len 的子序列归并
     i = 1;
     while( i+2*len<=L.length )
     {    //归并长 len 的两个子序列
          Merge( L, i, i+len-1, i+2*len-1);
          i = i+2*len;
     } // end while
     if ( i+len<=L.length )
          Merge( L, i, i+len-1, L.length );  // 归并最后两个子序列
} // MSort
```

算法 8.17 2 路归并非递归算法

```
void MergeSort(SqList &L)
{    // 对顺序表 L 作归并排序
     for( len=1; len<=L.length; len=2*len )
          MSort(L,len);
} // MergeSort
```

综上，对 2 路归并排序算法进行分析。

（1）时间复杂度分析：整个归并排序需进行$\log_2 n$趟归并，每一趟两两子序列归并，所有记录都参与比较和移位，因此其时间复杂度$T(n) = O(n \log_2 n)$。递归形式的算法在形式上较简洁，但实用性差。

（2）空间复杂度分析：实现归并排序需与待排记录数量相同的辅助空间，因此$S(n)=O(n)$。

（3）稳定性分析：由于归并排序是连续空间的移位，所以它是一种稳定的排序方法。

8.7 基数排序

前面介绍的排序算法都是基于关键字比较和移动记录这两种操作。**基数排序**（radix sort）是和前面所述各类排序方法完全不相同的一种排序方法，它不需要进行记录关键字间的比较。基数排序是一种借助多关键字排序的思想，将单关键字拆分成不同权重的“多关键字”，并对“多关键字”进行“分配”和“收集”，最终实现单关键字的排序的方法。

8.7.1 多关键字的排序

什么是多关键字排序？先看一个具体例子：每张扑克牌由多关键字，即花色和面值组成。假设“花色”的权重大于“面值”，其有序关系为：花色 ♦<♣<♥<♠，面值 2<3<…<K<A。

由此可得扑克牌中 52 张牌面的次序关系为：♦2<♦3<…<♦A<…<♠2<♠3<…<♠A。

将无序的扑克牌整理成上述次序的关系时，通常采用的办法有 2 种：

（1）先将 52 张牌按不同花色“分配”成 4 堆，使每一堆的牌均具有相同的花色，然后按面值分别对每一堆从小到大的顺序整理成有序，再将这 4 堆“收集”在一起。

（2）先将 52 张牌按不同面值“分配”成 13 堆，然后将这 13 堆牌自小至大“收集”在一起，再将 52 张牌按不同花色“分配”成 4 堆，然后将这 4 堆“收集”在一起。

以上就是两种多关键字的排序方法。一般情况下，假设每个记录R_i中含有d个关键字$(K_i^1,K_i^2,\dots,K_i^d)$，且权重$K_i^1 > K_i^2 > \cdots > K_i^d$，其中$K_i^1$为最主位关键字，$K_i^d$称为最次位关键字。为实现多关键字排序，通常有 2 种方法：

（1）先对最主位关键字进行排序，将序列分成若干子序列，然后分别就每个子序列的其次的关键字进行排序，再分成若干更小的子序列，以此重复，直至对最次位关键字进行排序之后得到最小的子序列，最后将所有子序列依次连接在一起成为一个有序序列。这种方法称为最高位优先（Most Significant Digit first）法，简称 MSD 法。

（2）先从最次位关键字开始进行排序，“收集”后再对高一位的关键字进行排序，以此重复，直至对最主位关键字进行排序后，“收集”成为一个有序序列。这种方法称为最低位优先(Least Significant Digit first）法，简称 LSD 法。

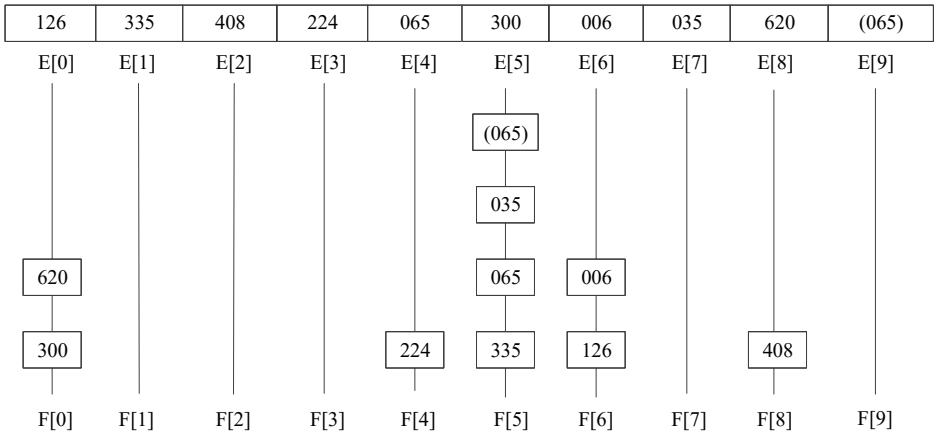
这两种排序方法的不同是：若按 MSD 进行排序，则必须将序列逐层分割成若干子序列，然后对各子序列分别进行排序；若按 LSD 进行排序，则不必分成子序列，而是将每次用于排序的关键字作为整体参加排序。

8.7.2 基数排序的实现

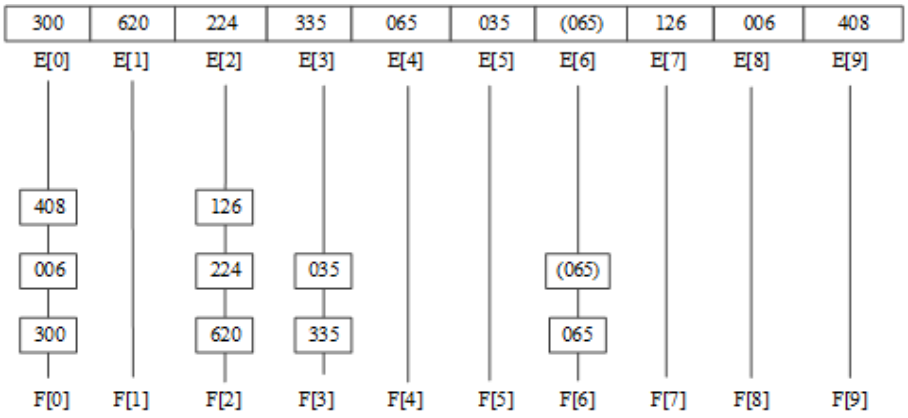
基数排序是借助“分配”和“收集”两种操作对单逻辑关键字进行排序的一种内部排序方法。

例如，若记录关键字是数值，其值都在$0 \le K \le 999$范围内，则可把记录关键字拆分成由位数（百位、十位、个位）组成的多关键字。由于每个位数取值范围小（0…9），所以按 LSD 进行排序更为方便：只需要从最低数位关键字起，按关键字的不同值将序列中记录“分配”到 10 个队列中后再“收集”之，如此重复 3 次。以这种方法实现的排序就称为基数排序，其中“基”指的是每个位数的取值范围，如果对应于数字和字母，则它们的“基”分别为“10”和“26”。

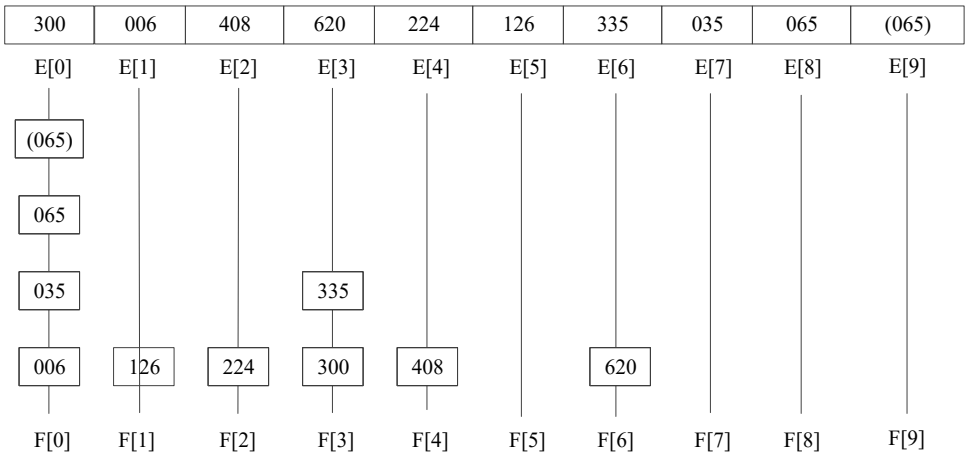
例 8.11 基数排序示例 以关键字序列{ 126，335，408，224，65，300，6，35，620，65}为例，基数排序的过程如图 8.11 所示。



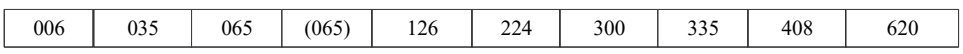
a）第 1 趟分配



b）第 1 趟收集和第 2 趟分配



c）第 2 趟收集和第 3 趟分配



d）第 3 趟收集得到最终结果

图 8.11　基数排序示例

基数排序算法思想如下（设记录关键字是数值，取值范围$0 \le key \le 999$）：

（1）将记录按个位的数值，对应“分配”到编号为 0~9 的 10 个队列中；

（2）按编号为 0~9 的次序“收集”10 个队列记录；

（3）按十位、百位重复上述过程，完成排序。

相应的基数排序的算法实现如下。

算法 8.18 基数排序算法

```
void RadixSort( SqList &L )
{   // 对关键字取值范围 0<=key<=999 的记录进行基数排序
    LinkQueue Q[10];     // 10 个辅助队列
    for ( i=0; i<10; i++)   InitQueue(Q[i]);
    radix = 1;
    for( k=1; k<=3; k++)
    {   // 分别对个位、十位、百位处理
        // 第 1 步：分配到合适的队列
        radix = radix*10;
        for( i=0; i<L.length; i++ )
        {   // 按基数进行“分配”进队列
            m = ( L.r[i].key % radix) / (radix/10);   // 取关键字的“位”
            EnQueue( Q[m], L.r[i] );                  // 按“位”进入对应的队列
        } // end for i
        // 第 2 步：将数据从队列收集到 L 中
        for( i=0, m=0; m<10; m++ )                    // 按次序进行“收集”队列
        {
            while( !QueueEmpty(Q[m]) )
            {
                DeQueue(Q[m], L.r[i]);
                i++;
            } // end while
        } // end for i
    } // end for k
}// RadixSort
```

由于每个队列“分配”的记录个数不确定，所以算法中最好采用链式队列。

综上，对基数排序算法进行分析。

（1）时间复杂度分析：基数排序是按位数对每个记录进行“分配”和“收集”的，所以其时间复杂度可以认为是$T(n)=O(d\cdot n)$，其中d为位数。

（2）空间复杂度分析：由于所有记录的“分配”都存储在链式队列中，并且每次用到了r个队头队尾指针，所以$S(n)=O(n+2r)$。

（3）稳定性分析：算法中采用队列进行“分配”和“收集”，而队列的先进先出特性保持了相同关键字记录的先后次序，因此该算法是稳定的。

8.8 各种内部排序方法的比较

本章内讨论的各种内部排序方法的算法时间复杂度、最坏情况、空间复杂度和稳定性如表 8.1 所示。

表 8.1 内部排序方法的比较

排序方法	平均时间	最坏情况	辅助存储	稳定性
直接插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序	$O(n^{1.5})$	$O(n^{1.5})$	$O(1)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(\log_2 n)$	不稳定
简单选择排序	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定

堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定
归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定
基数排序	$O(d\cdot n)$	$O(d\cdot n)$	$O(n+2r)$	稳定

通过比较分析，可以发现：

（1）从平均时间性能而言，快速排序最佳，其所需时间最少，但快速排序在最坏情况下的时间性能不如堆排序和归并排序。而后两者相比较的结果是，在n较大时，归并排序效率比堆排序高，但它所需的辅助存储量最多。

（2）表 8.1 中的“简单排序”包括直接插入排序，冒泡排序和简单选择排序。其中直接插入排序最简单，当序列中的记录“基本有序”或n值较小时，它是最佳的排序方法，因此常将它和其他的排序方法，如快速排序、归并排序等结合在一起。

（3）基数排序的时间复杂度是$O(d\cdot n)$。它最适用于n值很大而关键字较小的序列。若关键字也很大，而序列中大多数记录的“最高位关键字”均不同，则亦可先按“最高位关键字”不同将序列分成若干“小”的子序列，而后进行直接插入排序。

（4）从方法的稳定性来看，直接插入排序、冒泡排序是稳定的，简单选择排序不稳定。快速排序、堆排序和希尔排序等时间性能较好的排序方法都是不稳定的。归并排序、基数排序是稳定的。

8.9 小结

本章主要讨论了内部排序的五种方法：插入排序、交换排序、选择排序、归并排序和基数排序。针对每种方法都首先给出了一个具体的案例，然后讨论了算法思想、算法实现代码，并针对各算法进行了时间复杂度、空间复杂度和排序稳定性的分析，同时指出了可能的改进策略。

在针对插入排序、交换排序和选择排序的讨论中，先给出了经典的简单方法，其算法复杂度为$O(n^2)$；然后针对可能的优化策略，讨论了改进的方法，将算法复杂度降到$O(n^{1.5})$或$O(n\log n)$。特别地，基数排序根据关键字本身的特性，将基数排序算法的时间复杂度降到了$O(d\cdot n)$。

8.10 练习

1. 数列（46, 79, 56, 38, 40, 84）,采用快速排序（以最左侧元素为枢轴点），则第一次的划分结果为（）。

A. 38 46 79 56 40 84　　B. 38 79 56 46 40 84

C. 40 38 46 56 79 84　　D. 38 46 56 79 40 84

2. 使用递归的快速排序算法时，为了保证排序过程的时间复杂度不超过$O(nlogn)$，必须要做到（）。

A. 每次序列的划分应该在线性时间内完成

B. 每次归并的两个子序列长度接近

C. 每次归并在线性时间内完成

D. 以上都不对

3. 在快速排序时，要使最坏情况下时间复杂度为$O(nlogn)$，要对快速排序算法做（）修改。

A. 先排小子区间　　B. 先排大子区间

C. 划分的枢轴为三者取中　　D. 采用链表排序

4. 堆排序的时间复杂度是（）

A. $O(nlogn)$　　B.$O(n)$

C. $O(logn)$　　D.$O(1)$

5. 堆排序和快速排序，（）的空间复杂度高。

A. 堆排序　　B. 快速排序

C. 不确定

6. 二路归并排序的时间复杂度是（）。

A. $O(nlogn)$　　B. $O(n)$

C. $O(logn)$　　D. $O(1)$

7. 二路归并排序所需的辅助存储是（）。

A. $O(nlogn)$　　B. $O(n)$

C. $O(logn)$　　D. $O(1)$

8. 如果将所有中国人的按照生日（只考虑月、日）来排序，那么下面（）算法最快。

A. 归并排序　　B. 希尔排序

C. 快速排序　　D. 基数排序

9. 在下列指定的排序算法中，（）使用的附加空间与输入序列的长度及初始排列无关。

A. 锦标赛排序　　B. 快速排序

C. 基数排序　　D. 归并排序

10. 在下列指定的排序算法中，初始序列本身已排好序，（）算法是最快的。

A. 归并排序　　B. 直接插入排序

C. 简单选择排序　　D. 快速排序

11. 在下列指定的排序算法中，（）算法是不稳定的。

A. 起泡排序　　B. 直接插入排序

C. 快速排序

12. 若对 27 个元素只进行三趟多路归并排序，则选取的归并路数为（）

A. 2　　B. 3

C. 4　　D. 5

13. 编写函数，在基于单链表表示的待排序关键字序列上进行简单选择排序。

14. 如果只想在一个有n个元素的任意序列中，得到其中最小的$k(k<n)$个元素的部分排序序列，那么最好采用什么排序方法？为什么？

15. 设顺序结构线性表的元素值$K_1, K_2, \ldots, K_n$为考试成绩（整数 0~100），设计一个算法，将所有不及格（小于 60）的成绩放置在所有及格成绩之前，要求$T(n)=O(n)$，$S(n)=O(1)$。

16. 设有一个由一批需实时处理的数据元素组成集合S，实时处理开始后，每隔一 0.1 秒钟收到一个新的数据元素加入S。现要求在每次接收一个新元素之前，找出S中现有的最小元素并将其输

出（从S中删除）。试选择或构造一种适当的数据结构并设计一个算法，尽可能高效地完成上述任务。例如：S=(56,30,29,18,72,26,48,12,60,36)，新接受的数据为 32，46，16…。以此为例说明算法执行过程，并画出示意图。

17. 设有n个整数，存储在数组（int a[1..n]）中，并已构成小顶堆。编写函数，用整数e替换数组中第i个元素，并调整数组，使其还是小顶堆。

18. 排序算法比较

[问题描述]

生成 7 个样本文件，每个样本文件含 10000 个整数，其中第 1 个样本是正序，第 2 个样本是逆序，再利用随机函数产生 5 样本，每个样本有 10000 个随机整数。利用直接插入排序、希尔排序，冒泡排序、快速排序、选择排序、堆排序，归并排序、基数排序 8 种排序方法进行排序（结果为由小到大的顺序），并统计每一种排序算法对不同样本所耗费的时间。

[基本要求]

（1）原始数据存在文件中，用相同样本对不同算法进行测试；

（2）屏幕显示每种排序算法对不同样本所花的时间，即分别统计每个算法执行时的比较次数和移动次数。

8.11 延伸阅读

冒泡排序、插入排序和选择排序是一批最早应用于计算机的排序算法。它们都是日常生活中常用的排序算法，自然也被用于计算机中的排序问题。但这几种直观的排序方法的时间复杂度较高，平均情况时间复杂度均为$O(n^2)$。因此，如何使用计算机高效进行排序一直是被关注和研究的问题。Donald L. Shell 在 1959 年发明了 Shell 排序算法[1]，是对简单插入排序的一种改进，该方法排序时不需要额外空间，且与简单插入排序相比效率较高。

对于基于比较的排序算法，John von Neumann 在 1945 年基于分而治之思想首次发明了归并排序算法，将排序的时间复杂度降为$O(nlogn)$。Tony Hoare 在 1961 年发明了快速排序算法[2]，其平均情况时间复杂度也是$O(nlogn)$，直到今天也仍然是最优秀的排序算法之一。J. W. J. Williams 于 1964 年发明了堆排序算法[3]。堆排序算法是稳定的，最坏情况时间复杂度有保证，且只需要$O(1)$的额外空间。

计数排序和基数排序由 Harold H. Seward 于 1954 年发明[4]，它们是基于分配的排序方法，当待排序元素范围有限时，时间复杂度为$O(n)$，在特定问题中表现优秀。

近年来，随着对排序算法需求的增加，不断有新的排序算法出现，例如内省排序[5]和 Tim 排序。内省排序在保持了快速排序的高性能的同时，对快速排序可能会退化以及对少量元素排序效率较低的问题进行改进，已成为主流 C++标准库中的默认排序算法。Tim 排序则针对真实情况下数据种类繁多、结构复杂的情形进行优化，并广泛应用于 Python、Java 等编程语言中。

参考文献：

[1] SHELL D L. A high-speed sorting procedure[J]. Communications of the ACM, 1959, 2 (7): 30-32.
[2] HOARE C A R. Algorithm 64: quicksort[J], Communications of the ACM, 1961, 4(7): 321.
[3] WILLLAMS J W J. Algorithm 232: heapsort[J]. Communications of the ACM, 1964, 7(6): 347-348.

[4] SEWARD H H. Information sorting in the application of electronic digital computers to business operations[D]. Massachusetts Institute of Technology. Department of Electrical Engineering, 1954.
[5] MUSSER D R. Introspective sorting and selection algorithms[J]. Software: Practice and Experience, 1997, 27(8): 983-993